OpenClaw 记忆系统测试指令文档

文档概述

本文档为 OpenClaw 记忆系统的完整测试指令集，基于真实 Neo4j 图谱数据。所有指令可直接复制发送给 OpenClaw 执行。

图谱当前状态速查表

指标	数值	备注
节点总数	~3300	Entity ~1164, Strategy 72, MetaKnowledge 440, Archive ~2131
关系总数	~7016	全部为 RELATES_TO（待优化）
孤立节点比例	~73%	需要优化关系推断
实体类型	9 种	concept, technology, event, person, organization, location, product, project, other
已知问题	5 项	关系非语义化、重复实体、碎片实体、策略内容为空、孤立节点过多

A. 数据质量自检（Data Quality Audit）

测试 A1：图谱规模与结构统计

测试目的：获取图谱的基本统计数据，验证节点和关系的总数量，了解当前图谱规模。

复制以下指令发给 OpenClaw：

Plain Text

```
请查询 Neo4j 图谱并报告以下数据质量指标：

1. 节点总数统计：
   - 执行查询：MATCH (n) RETURN labels(n) as node_type, count(*) as count ORDER B
   - 报告各类型节点数量
```

```
2. 关系总数统计：
   - 执行查询：MATCH ()-[r]->() RETURN type(r) as rel_type, count(*) as count ORD
   - 报告各类型关系数量

3. 节点度数分析：
   - 执行查询：MATCH (n) WITH n, size((n)--()) as degree RETURN degree, count(*)
   - 报告度数分布（显示度数最高的 10 个区间）

请以表格形式呈现结果。
```

预期结果：

- 节点总数约 3300，其中 Entity 约 1164，Strategy 72，MetaKnowledge 440，Archive 约 2131
- 关系总数约 7016，全部为 RELATES_TO 类型
- 大部分节点度数较低（反映孤立节点问题）

判定标准：

- ✅ **通过**：数据与预期一致，能正确统计各类型节点和关系
- ❌ **不通过**：统计数据出现异常偏差（>10%）或无法执行查询

测试 A2：孤立节点检测

测试目的：识别没有任何关系的孤立节点，评估图谱连通性，为后续关系推断提供改进方向。

复制以下指令发给 OpenClaw：

Plain Text

```
请查询 Neo4j 图谱中的孤立节点：

1. 孤立节点总数：
   - 执行查询：MATCH (n) WHERE NOT (n)--() RETURN count(*) as isolated_count
   - 计算孤立节点比例：isolated_count / 总节点数

2. 孤立节点类型分布：
   - 执行查询：MATCH (n) WHERE NOT (n)--() RETURN labels(n) as node_type, count(*
   - 报告各类型孤立节点数量

3. 孤立实体样本（前 20 个）：
   - 执行查询：MATCH (n:Entity) WHERE NOT (n)--() RETURN n.name, n.entity_type, n
   - 列出孤立实体名称和类型

4. 孤立策略节点：
   - 执行查询：MATCH (n:Strategy) WHERE NOT (n)--() RETURN n.name, count(*) as co
```

```
    - 报告孤立策略数量

请以表格形式呈现结果，并分析孤立节点的主要原因。
```

预期结果：

- 孤立节点比例约 73%
- 孤立节点主要来自 Archive 类型（因为 Archive 是历史备份）
- 部分新创建的实体可能还未建立关系

判定标准：

- ✅ **通过**：孤立节点比例在 70%-80% 范围内，能识别主要来源
- ❌ **不通过**：孤立节点比例偏离预期过大（>10%）或无法分类统计

测试 A3：重复实体检测

测试目的：识别名称相同或相似的实体，检测是否存在应该合并的重复节点。

复制以下指令发给 OpenClaw：

Plain Text

```
请查询 Neo4j 图谱中的重复实体：

1. 完全相同名称的实体：
   - 执行查询：MATCH (n:Entity) WITH n.name as name, count(*) as cnt WHERE cnt >
   - 报告重复最多的实体名称

2. 重复实体详情（以"冥思日报"为例）：
   - 执行查询：MATCH (n:Entity) WHERE n.name = '冥思日报' RETURN n.name, n.entity_
   - 列出所有"冥思日报"节点的详细信息

3. 相似名称检测（编辑距离 < 2）：
   - 执行查询：MATCH (n:Entity) WITH n.name as name, collect(n) as nodes WHERE si
   - 报告可能需要合并的相似实体

4. 重复实体的关系情况：
   - 执行查询：MATCH (n:Entity)-[r]-(m) WHERE n.name = '冥思日报' RETURN n.name, t
   - 分析重复实体的关系是否指向相同的目标

请以表格形式呈现结果，并建议哪些实体应该被合并。
```

预期结果：

- 存在多个"冥思日报"副本

- 可能还有其他重复实体如"冥思"、"记忆系统"等
- 重复实体的关系指向可能不同

判定标准：

- ✅ **通过**：能识别出至少 3 个以上的重复实体，并提出合并建议
- ❌ **不通过**：无法识别重复实体或识别数量过少（<3）

测试 A4：碎片实体检测

测试目的：识别不完整、无意义或错误的实体节点（如"&&"、"无法"等），为数据清理提供依据。

复制以下指令发给 OpenClaw：

Plain Text

```
请查询 Neo4j 图谱中的碎片实体：

1. 短名称实体（可能是碎片）：
   - 执行查询：MATCH (n:Entity) WHERE length(n.name) <= 3 RETURN n.name, n.entity
   - 列出所有长度 ≤ 3 的实体名称

2. 特殊字符实体：
   - 执行查询：MATCH (n:Entity) WHERE n.name =~ '.*[&|!@#$%^*()\\-+=\\[\\]{}:;"\\
   - 列出包含特殊字符的实体

3. 常见碎片词汇检测：
   - 执行查询：MATCH (n:Entity) WHERE n.name IN ['&&', '无法', '明白', '嗯', '好的'
   - 检查是否存在这些常见碎片词汇

4. 无描述的实体：
   - 执行查询：MATCH (n:Entity) WHERE n.description IS NULL OR n.description = ''
   - 统计缺少描述的实体比例

请以表格形式呈现结果，并标记应该被删除的碎片实体。
```

预期结果：

- 存在"&&"、"无法"、"明白"等碎片词汇
- 短名称实体较多
- 大量实体缺少描述字段

判定标准：

- ✅ **通过**：能识别出至少 10 个以上的碎片实体，并提出清理建议

- ❌ **不通过**：无法识别碎片实体或识别数量过少（<5）

测试 A5：关系类型多样性分析

测试目的：检查关系类型的丰富度，评估冥思中"关系重标注"步骤的执行效果。

复制以下指令发给 OpenClaw：

```Plain Text
请查询 Neo4j 图谱中的关系类型多样性：

1. 关系类型统计：
   - 执行查询：MATCH ()-[r]->() RETURN type(r) as rel_type, count(*) as count ORD
   - 列出所有关系类型及其数量

2. 关系类型比例分析：
   - 执行查询：MATCH ()-[r]->() WITH type(r) as rel_type, count(*) as cnt RETURN
   - 计算各关系类型的占比

3. 关系属性检查：
   - 执行查询：MATCH ()-[r]->() WHERE r.description IS NOT NULL RETURN type(r) as
   - 统计有 description 的关系比例

4. 关系权重分布：
   - 执行查询：MATCH ()-[r]->() WHERE r.weight IS NOT NULL RETURN type(r) as rel_
   - 分析关系权重的分布情况

请以表格形式呈现结果，并评估关系类型多样性是否满足需求。
```

预期结果：

- 目前全部关系都是 RELATES_TO 类型（已知问题）
- 大部分关系缺少 description 和 weight 属性
- 这表明冥思的"关系重标注"步骤还未生效

判定标准：

- ✅ **通过**：能准确统计关系类型，发现 RELATES_TO 占比 100%
- ⚠️ **部分通过**：如果关系类型已开始多样化（RELATES_TO 占比 < 90%），说明关系重标注已开始生效
- ❌ **不通过**：无法统计关系类型或数据异常

测试 A6：实体激活度分布

测试目的：分析实体的激活度分布，识别高价值实体，为记忆召回优先级提供依据。

复制以下指令发给 OpenClaw：

Plain Text

```
请查询 Neo4j 图谱中的实体激活度分布：

1. 高激活度实体（TOP 20）：
   - 执行查询：MATCH (n:Entity) WHERE n.activation_level IS NOT NULL RETURN n.nam
   - 列出激活度最高的 20 个实体

2. 激活度分布统计：
   - 执行查询：MATCH (n:Entity) WHERE n.activation_level IS NOT NULL WITH n.activ
   - 统计各激活度区间的实体数量

3. 无激活度实体：
   - 执行查询：MATCH (n:Entity) WHERE n.activation_level IS NULL RETURN count(*)
   - 统计缺少激活度的实体比例

4. 激活度与度数的关联：
   - 执行查询：MATCH (n:Entity) WHERE n.activation_level IS NOT NULL WITH n, size
   - 分析高激活度实体的关系数量

请以表格形式呈现结果，并分析激活度与实体重要性的关联。
```

预期结果：

- 高激活度实体包括："冥思日报"(0.78)、"数据质量"(0.77)、"Neo4j"(0.76)、"记忆系统"(0.76)、"OpenClaw"(0.75)、"实体合并"(0.75)、"冥思"(0.75)
- 激活度范围 0.75-0.78
- 部分实体可能缺少激活度值

判定标准：

- ✅ **通过**：能识别出高激活度实体，激活度值在合理范围内（0.7-0.9）
- ❌ **不通过**：无法获取激活度数据或数据异常

B. 记忆写入测试（Memory Ingest）

测试 B1：新实体写入与验证

测试目的：验证 OpenClaw 能否正确将新信息写入图谱，检查写入的实体是否包含必要的元数据。

复制以下指令发给 OpenClaw：

Plain Text

```
请执行以下记忆写入测试：

1. 记住一条新信息：
   "OpenClaw 的记忆系统采用 Neo4j 图谱存储，支持多维度的知识表示，包括实体、关系、策略和元知

2. 写入后，请查询 Neo4j 图谱验证：
   - 查询新创建的实体：MATCH (n:Entity) WHERE n.created_at >= datetime.now() - dur
   - 检查是否创建了以下实体：
     * "记忆系统" 或 "Memory System"
     * "Neo4j"
     * "知识表示" 或 "Knowledge Representation"
     * "实体" 或 "Entity"
     * "关系" 或 "Relationship"
     * "策略" 或 "Strategy"
     * "元知识" 或 "Meta-Knowledge"

3. 验证写入的实体元数据：
   - 对于每个新创建的实体，检查：
     ✓ name 字段是否非空
     ✓ entity_type 是否为 9 种类型之一（concept, technology, event, person, organi
     ✓ description 是否有意义的文本
     ✓ activation_level 是否在 0-1 范围内
     ✓ created_at 是否为最近的时间戳

4. 报告结果：
   - 列出所有新创建的实体及其元数据
   - 标记哪些实体的元数据完整，哪些不完整
   - 如果有缺失的元数据，说明原因
```

预期结果：

- 应该创建 7 个新实体（或更多，如果进行了实体拆分）
- 每个实体都应该有 entity_type、description、activation_level
- created_at 应该是最近的时间戳

判定标准：

- ✅ **通过**：创建了至少 5 个新实体，每个实体都有完整的元数据（name, entity_type, description, activation_level, created_at）
- ⚠️ **部分通过**：创建了实体但元数据不完整（缺少 description 或 activation_level）
- ❌ **不通过**：未创建任何新实体或元数据严重缺失

测试 B2：新关系写入与语义化检查

测试目的：验证 OpenClaw 能否创建语义化的关系（而非全部 RELATES_TO），检查关系是否有正确的方向和属性。

复制以下指令发给 OpenClaw：

Plain Text

```
请执行以下关系写入测试：

1. 记住一条包含关系的新信息：
   "Neo4j 是一个图数据库技术，OpenClaw 的记忆系统使用 Neo4j 来存储知识图谱。"

2. 写入后，请查询 Neo4j 图谱验证关系：
   - 查询新创建的关系：MATCH (a)-[r]->(b) WHERE r.created_at >= datetime.now() - d
   - 检查是否创建了以下关系：
     * Neo4j -> (某种关系) -> 图数据库技术
     * Neo4j -> (某种关系) -> 记忆系统
     * 记忆系统 -> (某种关系) -> Neo4j
     * 记忆系统 -> (某种关系) -> 知识图谱

3. 验证关系的语义化程度：
   - 对于每个新创建的关系，检查：
     ✓ 关系类型是否为语义化类型（不是 RELATES_TO）
     ✓ 关系方向是否合理
     ✓ 关系是否有 description 字段
     ✓ 关系是否有 weight 字段（权重）

4. 报告结果：
   - 列出所有新创建的关系及其类型
   - 统计语义化关系的数量（非 RELATES_TO）
   - 如果全部都是 RELATES_TO，说明关系重标注还未生效
```

预期结果：

- 应该创建多个关系
- 理想情况下，关系类型应该包括 USES、STORES、IMPLEMENTS 等语义化类型
- 如果全部都是 RELATES_TO，说明冥思的"关系重标注"步骤还未生效

判定标准：

- ✅ **通过**：创建了至少 3 个关系，其中至少 50% 是语义化类型（非 RELATES_TO）
- ⚠️ **部分通过**：创建了关系但全部是 RELATES_TO，说明关系重标注还未生效
- ❌ **不通过**：未创建任何关系或无法验证

测试 B3：重复实体的自动合并

测试目的：验证 OpenClaw 在写入新信息时，是否能识别并合并重复实体。

复制以下指令发给 OpenClaw：

Plain Text

```
请执行以下重复实体合并测试：

1. 首先查询当前"冥思日报"的副本数量：
   - 执行查询：MATCH (n:Entity) WHERE n.name = '冥思日报' RETURN count(*) as count
   - 记录当前副本数量（预期为 > 1）

2. 记住一条关于"冥思日报"的新信息：
   "冥思日报是 OpenClaw 每日生成的知识摘要，包含高激活度实体的更新。"

3. 写入后，再次查询"冥思日报"的副本数量：
   - 执行查询：MATCH (n:Entity) WHERE n.name = '冥思日报' RETURN count(*) as count
   - 检查副本数量是否减少

4. 验证合并效果：
   - 如果副本数量减少，说明自动合并生效
   - 如果副本数量不变，说明自动合并还未实现
   - 查询合并后的"冥思日报"节点：MATCH (n:Entity) WHERE n.name = '冥思日报' RETURN n.
   - 检查合并后的节点是否有更多的关系（degree 增加）
```

预期结果：

- 如果自动合并生效，副本数量应该减少
- 合并后的节点应该有更多的关系
- 如果自动合并未生效，副本数量保持不变

判定标准：

- ✅ **通过**：副本数量减少，合并后的节点度数增加
- ⚠️ **部分通过**：副本数量不变，但新信息被正确写入
- ❌ **不通过**：无法验证或出现数据异常

C. 记忆召回测试（Memory Recall）

测试 C1：高激活度实体的自动召回

测试目的：验证 OpenClaw 能否基于对话上下文，自动召回高激活度实体，提升对话质量。

复制以下指令发给 OpenClaw：

Plain Text

```
请执行以下高激活度实体召回测试：

1. 首先查询当前的高激活度实体列表：
   - 执行查询：MATCH (n:Entity) WHERE n.activation_level IS NOT NULL RETURN n.nam
   - 记录前 10 个高激活度实体

2. 设计对话场景 1：提到高激活度实体
   用户输入："我想了解一下 Neo4j 的特点。"

   OpenClaw 应该：
   - 自动召回"Neo4j"实体（activation_level: 0.76）
   - 召回相关的实体如"图数据库"、"记忆系统"等
   - 在回答中提及这些相关实体

   验证：
   - 在 OpenClaw 的回答中是否提到了"Neo4j"的相关信息
   - 是否提到了其他高激活度实体（如"记忆系统"、"数据质量"等）

3. 设计对话场景 2：间接提到高激活度实体
   用户输入："我们的系统怎样保证数据的准确性？"

   OpenClaw 应该：
   - 自动召回"数据质量"实体（activation_level: 0.77）
   - 召回相关的实体和策略
   - 在回答中提及数据质量的相关内容

   验证：
   - 在 OpenClaw 的回答中是否提到了"数据质量"的相关信息
   - 是否提到了相关的实体或策略

4. 设计对话场景 3：提到"冥思"
   用户输入："冥思有什么作用？"

   OpenClaw 应该：
   - 自动召回"冥思"实体（activation_level: 0.75）
   - 召回"冥思日报"、"知识蒸馏"等相关实体
   - 在回答中详细解释冥思的功能

   验证：
   - 在 OpenClaw 的回答中是否准确解释了"冥思"的作用
   - 是否提到了冥思的相关步骤或产出

请报告每个场景中的召回效果，标记是否成功召回了高激活度实体。
```

预期结果：

- 场景 1：应该召回 Neo4j 及其相关实体
- 场景 2：应该召回数据质量及其相关实体
- 场景 3：应该召回冥思及其相关实体和策略

判定标准：

- ✅ **通过**：3 个场景中至少 2 个成功召回了高激活度实体及其相关信息
- ⚠️ **部分通过**：能召回实体但相关信息不完整
- ❌ **不通过**：无法召回高激活度实体或召回效果差

测试 C2：跨话题关联召回

测试目的：验证 OpenClaw 能否识别不同话题之间的关联，进行跨域知识召回。

复制以下指令发给 OpenClaw：

Plain Text

```
请执行以下跨话题关联召回测试：

1. 首先查询实体之间的关系网络：
   - 执行查询：MATCH (n:Entity {name: 'Neo4j'})-[r]-(m) RETURN n.name, type(r), m
   - 查看 Neo4j 相关的实体有哪些

2. 设计对话场景 1：从技术到应用的关联
   用户输入："Neo4j 在我们的项目中有什么应用？"

   OpenClaw 应该：
   - 召回 Neo4j 实体
   - 通过关系网络找到相关的应用场景（如"记忆系统"、"知识图谱"等）
   - 在回答中连接技术和应用

   验证：
   - 回答是否提到了 Neo4j 的具体应用
   - 是否提到了"记忆系统"、"知识图谱"等相关概念

3. 设计对话场景 2：从问题到解决方案的关联
   用户输入："我们的数据质量问题怎样解决？"

   OpenClaw 应该：
   - 召回"数据质量"实体
   - 通过关系网络找到相关的解决方案（如"冥思"、"数据清理"等）
   - 在回答中提供具体的解决方案
```

```
    验证：
    - 回答是否提到了具体的解决方案
    - 是否提到了"冥思"等相关工具或流程

4. 设计对话场景 3：从概念到实现的关联
    用户输入："实体合并的目的是什么？"

    OpenClaw 应该：
    - 召回"实体合并"实体（activation_level: 0.75）
    - 通过关系网络找到相关的概念（如"重复实体"、"数据质量"等）
    - 在回答中解释实体合并的目的和意义

    验证：
    - 回答是否清楚地解释了实体合并的目的
    - 是否提到了相关的概念和背景

请报告每个场景中的关联召回效果，标记是否成功进行了跨话题关联。
```

预期结果：

- 场景 1：应该将 Neo4j 与记忆系统、知识图谱等应用关联
- 场景 2：应该将数据质量与冥思、数据清理等解决方案关联
- 场景 3：应该将实体合并与重复实体、数据质量等概念关联

判定标准：

- ✅ **通过**：3 个场景中至少 2 个成功进行了跨话题关联
- ⚠️ **部分通过**：能进行关联但深度不足
- ❌ **不通过**：无法进行跨话题关联或关联错误

测试 C3：策略召回与推荐

测试目的：验证 OpenClaw 能否基于对话场景，召回和推荐相关的策略。

复制以下指令发给 OpenClaw：

Plain Text

```
请执行以下策略召回测试：

1. 首先查询当前的策略节点：
    - 执行查询：MATCH (n:Strategy) RETURN n.name, n.description, n.content LIMIT 2
    - 了解当前有哪些策略

2. 设计对话场景 1：数据质量问题
    用户输入："我们发现图谱中有很多重复实体，应该怎样处理？"
```

```
   OpenClaw 应该：
   - 识别问题类型：数据质量 -> 重复实体
   - 召回相关的策略（如"实体合并策略"、"去重策略"等）
   - 在回答中推荐具体的解决方案

   验证：
   - 是否推荐了相关的策略
   - 策略推荐是否合理和有针对性

3. 设计对话场景 2：关系优化问题
   用户输入："我们的图谱中关系都是 RELATES_TO，怎样进行关系重标注？"

   OpenClaw 应该：
   - 识别问题类型：关系优化 -> 关系重标注
   - 召回相关的策略（如"关系重标注策略"、"语义化关系策略"等）
   - 在回答中推荐冥思流程中的"关系重标注"步骤

   验证：
   - 是否推荐了"关系重标注"策略
   - 是否提到了冥思流程

4. 设计对话场景 3：知识蒸馏问题
   用户输入："怎样从现有的知识图谱中提取高质量的元知识？"

   OpenClaw 应该：
   - 识别问题类型：知识提取 -> 知识蒸馏
   - 召回相关的策略（如"知识蒸馏策略"、"元知识提取策略"等）
   - 在回答中推荐冥思流程中的"知识蒸馏"步骤

   验证：
   - 是否推荐了"知识蒸馏"策略
   - 是否提到了元知识的生成过程

请报告每个场景中的策略召回效果，标记是否成功推荐了相关策略。
```

预期结果：

- 场景 1：应该推荐实体合并或去重相关的策略
- 场景 2：应该推荐关系重标注策略或冥思流程
- 场景 3：应该推荐知识蒸馏策略或元知识提取方法

判定标准：

- ✅ **通过**：3 个场景中至少 2 个成功推荐了相关策略
- ⚠️ **部分通过**：能推荐策略但不够精准

- ❌ **不通过**：无法推荐策略或推荐错误

D. 冥思效果验证（Meditation Pipeline Validation）

测试 D1：冥思流程 Dry-Run

测试目的：触发一次冥思 dry-run，检查各步骤是否正常执行，验证冥思流程的完整性。

复制以下指令发给 OpenClaw：

Plain Text

```
请执行以下冥思 dry-run 测试：

1. 触发冥思流程（dry-run 模式）：
   - 使用 POST 请求到 http://127.0.0.1:18900/meditation/trigger
   - 请求体：{"dry_run": true, "mode": "full"}
   - 记录返回的 meditation_id 和状态

2. 等待冥思流程完成（最多等待 5 分钟）：
   - 每 10 秒查询一次冥思状态
   - 查询接口：GET http://127.0.0.1:18900/meditation/{meditation_id}/status
   - 记录各步骤的执行时间和状态

3. 验证冥思的 9 个步骤是否都正常执行：
   - Step 0: 信念冲突检测 - 检查是否检测到冲突
   - Step 1: 注意力优先级排序 - 检查是否排序了高激活度实体
   - Step 2: 快照备份 - 检查是否创建了备份
   - Step 3.1: 同义词判断 - 检查是否识别了同义词
   - Step 3.2: 截断修复 - 检查是否修复了碎片实体
   - Step 3.3: 元数据补充 - 检查是否补充了缺失的元数据
   - Step 4.1: 关系重标注 - 检查是否重标注了关系类型
   - Step 4.2: 隐含关系推断 - 检查是否推断了新关系
   - Step 5: 权重强化与衰减 - 检查是否调整了权重
   - Step 6: 知识蒸馏 - 检查是否生成了元知识
   - Step 6.5: 策略蒸馏 - 检查是否生成了策略
   - Step 6.6: 策略进化 - 检查是否进化了策略
   - Step 7: 解锁节点 - 检查是否解锁了节点
   - Step 7.5: self_evolution_pipeline - 检查是否执行了自进化

4. 报告结果：
   - 列出所有步骤的执行状态（成功/失败/跳过）
   - 对于失败的步骤，记录错误信息
   - 统计成功执行的步骤数量

5. 由于是 dry-run，验证图谱是否未被修改：
```

```
    - 比较 dry-run 前后的节点和关系数量
    - 确认 dry-run 不会修改实际数据
```

预期结果：

- 冥思流程应该能成功启动
- 大部分步骤应该能正常执行
- dry-run 模式不应该修改实际数据

判定标准：

- ✅ **通过**：冥思流程成功启动，至少 80% 的步骤正常执行，dry-run 不修改数据
- ⚠️ **部分通过**：冥思流程启动但部分步骤失败，或 dry-run 模式有问题
- ❌ **不通过**：冥思流程无法启动或大部分步骤失败

测试 D2：同义词合并建议

测试目的：验证冥思的"同义词判断"步骤是否能正确识别应该合并的同义词实体。

复制以下指令发给 OpenClaw：

Plain Text

```
请执行以下同义词合并测试：

1. 查询当前的重复实体（可能是同义词）：
   - 执行查询：MATCH (n:Entity) WITH n.name as name, count(*) as cnt WHERE cnt >
   - 列出重复最多的实体名称

2. 触发冥思流程并关注 Step 3.1（同义词判断）：
   - 使用 POST 请求到 http://127.0.0.1:18900/meditation/trigger
   - 请求体：{"dry_run": false, "focus_step": "3.1"}
   - 等待冥思完成

3. 查询冥思的建议（同义词合并建议 ）：
   - 执行查询：MATCH (n:MeditationSuggestion) WHERE n.type = 'SYNONYM_MERGE' RETU
   - 列出所有同义词合并建议

4. 验证建议的合理性：
   - 对于每个建议，检查：
     ✓ source_entity 和 target_entity 是否真的是同义词
     ✓ confidence 分数是否合理（0-1）
     ✓ reason 是否有说服力

5. 执行建议的合并（可选）：
   - 对于 confidence > 0.8 的建议，执行合并
```

```
    - 合并后验证图谱的变化

请报告同义词合并建议的数量和质量。
```

预期结果：

- 应该识别出"冥思日报"的多个副本
- 应该提出合并建议
- 建议的 confidence 应该较高（>0.8）

判定标准：

- ✅ **通过**：识别出至少 3 个同义词合并建议，confidence > 0.8
- ⚠️ **部分通过**：识别出建议但 confidence 较低或建议不准确
- ❌ **不通过**：无法识别同义词或建议错误

测试 D3：关系重标注建议

测试目的：验证冥思的"关系重标注"步骤是否能将 RELATES_TO 关系重标注为语义化类型。

复制以下指令发给 OpenClaw：

Plain Text

```
请执行以下关系重标注测试：

1. 查询当前的关系类型分布：
   - 执行查询：MATCH ()-[r]->() RETURN type(r) as rel_type, count(*) as count ORD
   - 记录 RELATES_TO 的数量（预期为 100%）

2. 触发冥思流程并关注 Step 4.1（关系重标注）：
   - 使用 POST 请求到 http://127.0.0.1:18900/meditation/trigger
   - 请求体：{"dry_run": false, "focus_step": "4.1"}
   - 等待冥思完成

3. 查询冥思的建议（关系重标注建议 ）：
   - 执行查询：MATCH (n:MeditationSuggestion) WHERE n.type = 'RELATIONSHIP_RETYPE
   - 列出所有关系重标注建议

4. 验证建议的合理性：
   - 对于每个建议，检查：
     ✓ old_type 是否为 RELATES_TO
     ✓ new_type 是否为语义化类型（如 USES, IMPLEMENTS, STORES 等）
     ✓ confidence 分数是否合理（0-1）
     ✓ reason 是否有说服力
```

```
5. 执行建议的重标注（可选）：
   - 对于 confidence > 0.8 的建议，执行重标注
   - 重标注后再次查询关系类型分布
   - 验证 RELATES_TO 的比例是否下降

请报告关系重标注建议的数量、质量和执行效果。
```

预期结果：

- 应该识别出大量的 RELATES_TO 关系
- 应该提出重标注建议，将其转换为语义化类型
- 重标注后，RELATES_TO 的比例应该下降

判定标准：

- ✅ **通过**：识别出至少 10 个关系重标注建议，confidence > 0.8，执行后 RELATES_TO 比例下降 > 10%
- ⚠️ **部分通过**：识别出建议但数量较少或 confidence 较低
- ❌ **不通过**：无法识别关系或建议错误

测试 D4：知识蒸馏产出质量

测试目的：验证冥思的"知识蒸馏"步骤是否能生成高质量的元知识。

复制以下指令发给 OpenClaw：

Plain Text

```
请执行以下知识蒸馏测试：

1. 查询当前的元知识节点数量：
   - 执行查询：MATCH (n:MetaKnowledge) RETURN count(*) as count
   - 记录当前元知识数量（预期为 ~440）

2. 触发冥思流程并关注 Step 6（知识蒸馏）：
   - 使用 POST 请求到 http://127.0.0.1:18900/meditation/trigger
   - 请求体：{"dry_run": false, "focus_step": "6"}
   - 等待冥思完成

3. 查询新生成的元知识：
   - 执行查询：MATCH (n:MetaKnowledge ) WHERE n.created_at >= datetime.now() - du
   - 列出最新生成的元知识

4. 验证元知识的质量：
   - 对于每个新生成的元知识，检查：
     ✓ name 是否清晰简洁
```

```
      ✓ content 是否有意义和有用
      ✓ source_entities 是否正确指向源实体
      ✓ confidence 是否合理（0-1）

5. 分析元知识的主题分布：
   - 执行查询：MATCH (n:MetaKnowledge) WHERE n.created_at >= datetime.now() - dur
   - 统计各主题的元知识数量

请报告新生成的元知识数量、质量和主题分布。
```

预期结果：

- 应该生成新的元知识节点
- 元知识应该有清晰的 name 和 content
- 元知识应该正确指向源实体
- confidence 应该较高（>0.7）

判定标准：

- ✅ **通过**：生成至少 5 个新元知识，每个都有完整的元数据和较高的 confidence
- ⚠️ **部分通过**：生成元知识但质量不够高或数量较少
- ❌ **不通过**：无法生成元知识或质量很差

E. 端到端体验测试（End-to-End Experience Test）

测试 E1：对话场景 1 - 技术咨询

测试目的：在真实的对话场景中，验证记忆系统对对话质量的提升。

复制以下指令发给 OpenClaw：

Plain Text

```
请执行以下对话场景测试：

场景描述：用户向 OpenClaw 咨询关于记忆系统的技术问题。

对话流程：
1. 用户输入："我想了解一下 OpenClaw 的记忆系统是如何工作的。"

   OpenClaw 应该：
   - 自动召回"记忆系统"、"Neo4j"、"知识图谱"等高激活度实体
   - 召回相关的概念和技术
   - 提供清晰的解释
```

```
2. 用户追问："它和传统的数据库有什么区别？"

   OpenClaw 应该：
   - 基于前一个对话的上下文
   - 召回"Neo4j"、"图数据库"等相关实体
   - 进行对比分析

3. 用户再追问："那冥思有什么作用？"

   OpenClaw 应该：
   - 自动召回"冥思"实体及其 9 个步骤
   - 解释冥思如何优化记忆系统
   - 提供具体的例子

验证标准：
- 每个回答是否都准确召回了相关实体
- 回答是否连贯且上下文相关
- 是否提到了高激活度实体
- 回答的信息完整度和准确度

请报告：
- 每个回答中召回的实体列表
- 回答的质量评分（1-5 分）
- 是否成功提升了对话质量
```

预期结果：

- 第一个回答应该提到记忆系统、Neo4j、知识图谱等
- 第二个回答应该进行技术对比
- 第三个回答应该详细解释冥思

判定标准：

- ✅ **通过**：3 个回答都准确召回了相关实体，质量评分 ≥ 4 分
- ⚠️ **部分通过**：回答基本准确但召回不够完整
- ❌ **不通过**：回答质量差或无法召回实体

测试 E2：对话场景 2 - 问题诊断

测试目的：在问题诊断场景中，验证记忆系统能否帮助 OpenClaw 快速定位问题并提供解决方案。

复制以下指令发给 OpenClaw：

Plain Text

```
请执行以下对话场景测试：

场景描述：用户向 OpenClaw 报告一个数据质量问题。

对话流程：
1. 用户输入："我们发现图谱中有很多重复的实体，这会影响查询效率。"

   OpenClaw 应该：
   - 自动召回"数据质量"、"重复实体"、"实体合并"等相关实体
   - 识别问题的严重性
   - 提供初步的诊断

2. 用户追问："这个问题怎样解决？"

   OpenClaw 应该：
   - 召回相关的解决方案和策略
   - 提到"冥思"流程中的"同义词判断"和"实体合并"步骤
   - 提供具体的操作建议

3. 用户再追问："需要多长时间才能修复？"

   OpenClaw 应该：
   - 基于图谱规模和重复实体数量进行估算
   - 提供时间预估
   - 解释影响因素

验证标准：
- 是否准确识别了问题类型
- 是否提供了合理的解决方案
- 是否提到了相关的冥思步骤
- 回答的专业性和可操作性

请报告：
- 每个回答中召回的实体和策略列表
- 问题诊断的准确度（1-5 分）
- 解决方案的可操作性（1-5 分）
```

预期结果：

- 第一个回答应该识别出数据质量问题
- 第二个回答应该提到冥思的实体合并功能
- 第三个回答应该提供时间估算

判定标准：

- ✅ **通过**：问题诊断准确，解决方案可操作，评分 ≥ 4 分

- ⚠️ **部分通过**：诊断基本准确但方案不够完整
- ❌ **不通过**：诊断错误或方案不可行

测试 E3：对话场景 3 - 知识探索

测试目的：在知识探索场景中，验证记忆系统能否帮助用户发现新的知识关联。

复制以下指令发给 OpenClaw：

Plain Text

```
请执行以下对话场景测试：

场景描述：用户向 OpenClaw 探索关于"数据质量"的相关知识。

对话流程：
1. 用户输入："数据质量涉及哪些方面？"

   OpenClaw 应该：
   - 自动召回"数据质量"实体
   - 通过关系网络找到相关的维度和方面
   - 提供全面的列表

2. 用户追问："这些方面中哪个最重要？"

   OpenClaw 应该：
   - 基于实体的激活度进行排序
   - 提供基于图谱数据的排名
   - 解释排名的原因

3. 用户再追问："怎样改进这个方面？"

   OpenClaw 应该：
   - 召回相关的改进策略
   - 提到具体的工具或方法
   - 提供可操作的建议

验证标准：
- 是否发现了数据质量的多个方面
- 排序是否基于实体的重要性
- 改进建议是否具体和可操作
- 知识探索的深度和广度

请报告：
- 每个回答中发现的知识关联数量
```

```
- 知识探索的深度评分（1-5 分）
- 是否发现了新的知识关联
```

预期结果：

- 第一个回答应该列出数据质量的多个方面
- 第二个回答应该基于激活度进行排序
- 第三个回答应该提供具体的改进建议

判定标准：

- ✅ **通过**：发现至少 5 个知识关联，深度评分 ≥ 4 分
- ⚠️ **部分通过**：发现关联但深度不足
- ❌ **不通过**：无法发现知识关联

测试 E4：对话场景 4 - 上下文连贯性

测试目的：验证记忆系统能否维持长对话中的上下文连贯性。

复制以下指令发给 OpenClaw：

Plain Text

```
请执行以下对话场景测试：

场景描述：用户与 OpenClaw 进行一个较长的多轮对话，测试上下文连贯性。

对话流程（共 5 轮）：
1. 用户输入："Neo4j 是什么？"
   记录 OpenClaw 的回答

2. 用户输入："它在我们的项目中有什么应用？"
   检查是否基于第 1 轮的上下文进行回答

3. 用户输入："我们现在遇到了性能问题，怎样优化？"
   检查是否基于前两轮的上下文（Neo4j + 项目应用）进行回答

4. 用户输入："这个优化方案需要修改现有的数据吗？"
   检查是否基于前三轮的上下文进行回答

5. 用户输入："那冥思能帮助我们吗？"
   检查是否能将冥思与前面的问题（Neo4j、性能优化、数据修改）关联

验证标准：
- 每个回答是否都基于前面的对话上下文
- 是否能正确追踪对话的主题演变
```

```
- 是否能在不同的对话阶段召回相关的历史信息
- 上下文连贯性

请报告：
- 每个回答是否都基于上下文
- 上下文连贯性评分（1-5 分）
- 是否出现了上下文混淆或遗忘
```

预期结果：

- 每个回答都应该基于前面的对话
- 应该能正确追踪对话主题
- 应该能在适当的时候回顾前面的信息

判定标准：

- ✅ **通过**：5 个回答都基于上下文，连贯性评分 ≥ 4 分
- ⚠️ **部分通过**：大部分回答基于上下文但有偶发的混淆
- ❌ **不通过**：多个回答无法基于上下文或连贯性很差

测试 E5：对话场景 5 - 个性化推荐

测试目的：验证记忆系统能否基于用户的历史交互，提供个性化的推荐。

复制以下指令发给 OpenClaw：

Plain Text

```
请执行以下对话场景测试：

场景描述：用户与 OpenClaw 进行多次交互后，系统应该能学习用户的偏好并提供个性化推荐。

对话流程：
1. 用户输入："我对数据质量很感兴趣，能否推荐一些相关的资源或话题？"

   OpenClaw 应该：
   - 基于用户的历史交互（如前面的对话）
   - 了解用户对"数据质量"的关注
   - 提供个性化的推荐

2. 用户输入："除了数据质量，还有什么其他相关的话题我应该了解？"

   OpenClaw 应该：
   - 基于用户的兴趣（数据质量）
   - 通过关系网络找到相关的话题
   - 提供个性化的推荐
```

```
3. 用户输入："这些话题中，哪个最适合我现在学习？"

   OpenClaw 应该：
   - 基于用户的学习进度和兴趣
   - 提供个性化的学习路径建议
   - 解释推荐的原因

验证标准：
- 是否基于用户的历史交互进行推荐
- 推荐是否个性化和相关
- 推荐是否有合理的优先级排序
- 个性化推荐的质量

请报告：
- 每个推荐的相关性评分（1-5 分）
- 个性化程度评分（1-5 分）
- 是否成功学习了用户的偏好
```

预期结果：

- 推荐应该与用户的历史交互相关
- 推荐应该有合理的优先级
- 推荐应该帮助用户进行学习

判定标准：

- ✅ **通过**：推荐相关性 ≥ 4 分，个性化程度 ≥ 4 分
- ⚠️ **部分通过**：推荐基本相关但个性化程度不足
- ❌ **不通过**：推荐不相关或无法个性化

测试总结表

测试类别	测试项	通过/不通过	备注
A. 数据质量	A1: 图谱规模统计		
	A2: 孤立节点检测		
	A3: 重复实体检测		
	A4: 碎片实体检测		
	A5: 关系类型多样性		

	A6: 实体激活度分布		
B. 记忆写入	B1: 新实体写入		
	B2: 新关系写入		
	B3: 重复实体合并		
C. 记忆召回	C1: 高激活度实体召回		
	C2: 跨话题关联召回		
	C3: 策略召回推荐		
D. 冥思效果	D1: 冥思流程 Dry-Run		
	D2: 同义词合并建议		
	D3: 关系重标注建议		
	D4: 知识蒸馏质量		
E. 端到端体验	E1: 技术咨询场景		
	E2: 问题诊断场景		
	E3: 知识探索场景		
	E4: 上下文连贯性		
	E5: 个性化推荐		

快速参考：常用 Cypher 查询

实体相关查询

Plain Text

```
# 查询所有实体
MATCH (n:Entity) RETURN n.name, n.entity_type, n.activation_level LIMIT 20

# 查询高激活度实体
MATCH (n:Entity) WHERE n.activation_level IS NOT NULL
RETURN n.name, n.entity_type, n.activation_level
ORDER BY n.activation_level DESC LIMIT 20
```

```
# 查询孤立实体
MATCH (n:Entity) WHERE NOT (n)--()
RETURN n.name, n.entity_type, count(*) as count

# 查询重复实体
MATCH (n:Entity) WITH n.name as name, count(*) as cnt
WHERE cnt > 1
RETURN name, cnt ORDER BY cnt DESC LIMIT 20

# 查询特定实体的关系
MATCH (n:Entity {name: '冥思'})-[r]-(m)
RETURN n.name, type(r), m.name, m.activation_level
ORDER BY m.activation_level DESC LIMIT 20
```

关系相关查询

Plain Text

```
# 查询所有关系类型
MATCH ()-[r]->()
RETURN type(r) as rel_type, count(*) as count
ORDER BY count DESC

# 查询 RELATES_TO 关系数量
MATCH ()-[r:RELATES_TO]->()
RETURN count(*) as relates_to_count

# 查询有 description 的关系
MATCH ()-[r]->() WHERE r.description IS NOT NULL
RETURN type(r), count(*) as count

# 查询关系权重分布
MATCH ()-[r]->() WHERE r.weight IS NOT NULL
RETURN type(r), min(r.weight), max(r.weight), avg(r.weight)
```

元知识相关查询

Plain Text

```
# 查询所有元知识
MATCH (n:MetaKnowledge)
RETURN n.name, n.content LIMIT 20

# 查询最近创建的元知识
MATCH (n:MetaKnowledge) WHERE n.created_at IS NOT NULL
RETURN n.name, n.content, n.created_at
```

```
ORDER BY n.created_at DESC LIMIT 20

# 查询有 source_entities 的元知识
MATCH (n:MetaKnowledge) WHERE n.source_entities IS NOT NULL
RETURN count(*) as count
```

策略相关查询

Plain Text

```
# 查询所有策略
MATCH (n:Strategy)
RETURN n.name, n.description, n.content LIMIT 20

# 查询有 content 的策略
MATCH (n:Strategy) WHERE n.content IS NOT NULL
RETURN n.name, n.content LIMIT 20

# 查询孤立策略
MATCH (n:Strategy) WHERE NOT (n)--()
RETURN n.name, count(*) as count
```

测试执行建议

执行顺序

建议按以下顺序执行测试，以获得最佳结果：

1. **第一阶段：数据质量基线**（A 类测试）
   - 首先执行 A1-A6，了解图谱的当前状态
   - 这为后续测试提供基准数据
2. **第二阶段：写入能力**（B 类测试）
   - 执行 B1-B3，验证记忆写入功能
   - 这会向图谱添加新数据，为后续测试提供素材
3. **第三阶段：召回能力**（C 类测试）
   - 执行 C1-C3，验证记忆召回功能
   - 这依赖于前面的写入测试
4. **第四阶段：冥思效果**（D 类测试）
   - 执行 D1-D4，验证冥思流程

- 这会优化图谱数据

5. **第五阶段：端到端体验**（E 类测试）
   - 执行 E1-E5，验证整体的对话体验
   - 这是最终的综合测试

每个测试的时间估算

测试	预计时间
A1-A6	30-45 分钟
B1-B3	20-30 分钟
C1-C3	30-45 分钟
D1-D4	45-60 分钟（冥思可能需要较长时间）
E1-E5	30-45 分钟
总计	**2.5-4 小时**

测试中的常见问题

Q1: 如何访问 Neo4j 图谱?

A: 使用提供的 HTTP API 地址和认证信息：

```Plain Text
URL: http://127.0.0.1:7474/db/neo4j/tx/commit
认证: neo4j / reflection123
```

Q2: 冥思流程需要多长时间?

A: 取决于图谱规模和选定的步骤 ，通常 5-30 分钟。

Q3: 如何判断测试是否通过?

A: 参考每个测试的"判定标准"部分。

Q4: 如果测试失败怎么办?

A: 记录错误信息，检查系统日志，然后重新执行测试。

附录：已知问题和改进方向

当前已知问题

1. **关系非语义化**：所有关系都是 RELATES_TO，缺乏语义信息
   - 改进方向：执行冥思的"关系重标注"步骤
2. **重复实体**：存在多个"冥思日报"等重复节点
   - 改进方向：执行冥思的"同义词判断"步骤
3. **碎片实体**：存在"&&"、"无法"等无意义的实体
   - 改进方向：执行冥思的"截断修复"步骤
4. **策略内容为空**：72 个策略节点大部分没有 content
   - 改进方向：执行冥思的"策略蒸馏"步骤
5. **孤立节点过多**：约 73% 的节点没有关系
   - 改进方向：执行冥思的"隐含关系推断"步骤

改进优先级

优先级	问题	预期改进效果
P0	孤立节点过多	提高图谱连通性，增加召回成功率
P1	关系非语义化	提高关系质量，增加知识表示的丰富度
P2	重复实体	提高数据质量，减少冗余
P3	碎片实体	提高数据清洁度
P4	策略内容为空	增强策略推荐功能

文档版本：v1.0

最后更新：2026-04-05

适用系统：OpenClaw Memory System v1.0+